

Journal

Week 1, Assignment 6

Hooria Zahoor | BS Artificial Intelligence, The University of Faisalabad

Why did you choose AI Engineering?

I chose AI Engineering because it sits right at the intersection of two things I care about: building software that actually works end-to-end, and working with systems that can reason over language. I've spent most of my degree on the modeling side — fine-tuning transformers, running experiments, evaluating metrics — but I kept noticing that the gap between a working model in a notebook and a working product in someone's hands is enormous, and almost nobody teaches that gap explicitly. AI Engineering is exactly that missing layer: prompt design, tool calling, memory, error handling, deployment — the practical craft of turning a model into something reliable. This fellowship's Week 1 theme, 'From AI User to AI Builder,' captures precisely why I'm here.

What type of AI products do you want to build?

I'm most drawn to AI products that genuinely support people rather than quietly making them dependent — tools that inform a decision instead of making it for you, or that step in during moments that actually matter. My final year project is a mental health conversational agent with crisis detection, and working on it has shaped how I think about AI product design in general: the goal isn't just an impressive demo, it's a system a real person can trust when it counts. Beyond that specific project, I'm interested in NLP-driven agents that combine solid language understanding with explainability — I want the people using what I build to understand **why** it said what it said, not just receive an answer.

What was your biggest learning this week?

- AI Engineering is mostly not about the model itself. Building the AI Workspace app, the actual API call was maybe 10 lines of code — everything else was system prompts, session state, error handling, and UI logic.
- A model's behavior changes dramatically with how a prompt is framed — seeing role prompting and Chain-of-Thought side-by-side in my experiments made concepts that felt abstract in the readings suddenly concrete.
- Small prompt changes (adding a role, a format constraint, or an audience) had a bigger effect on output quality than I expected — specificity matters more than length.
- Debugging a framework (like Streamlit's widget state model) is a skill in itself, separate from knowing its API — understanding **why** something breaks is more valuable than memorizing syntax.
- Free-tier infrastructure choices (like switching from OpenAI to Gemini) can be just as important as the code itself when building something that has to actually be usable, not just functional in theory.

What challenges did you face? How did you solve them?

A few real challenges came up while building the app:

- Prompt templates weren't working. I initially built the template buttons to prefill the chat box, but they silently did nothing when clicked. I eventually traced it to how Streamlit manages widget state across reruns — once a widget has a key, it ignores the `value=` parameter on later runs. I fixed it using a versioned-key pattern, where clicking a template generates a fresh widget key with the new text pre-loaded. It was a useful lesson in understanding a framework's execution model, not just its API surface.
- A CSS layering bug made the whole app disappear. While adding an animated background, an overly strong blur and incorrect z-index stacking caused the entire interface — sidebar, chat, everything — to be hidden behind a wash of color. I fixed it using ``isolation: isolate`` and a negative z-index on the background layer, which taught me more about how stacking contexts actually work than any tutorial had.
- Environment and API-cost confusion. I originally set the app up with the OpenAI API before realizing it required paid billing with no free tier. I switched to Google Gemini's free, OpenAI-compatible endpoint instead, which meant only changing the base URL and API key — the rest of the code stayed the same. I also ran into some virtual-environment and port-conflict issues while testing locally, which I resolved by being more careful about closing old Streamlit sessions before starting new ones.

Across all three, the pattern was the same: the fix was rarely about writing more code, it was about understanding **why** the existing code was behaving the way it was.

What are your goals for Week 2?

- Implement real tool/function calling instead of just chatting — connect the agent to at least one external tool (e.g., a calculator, a search function, or a simple API).
- Add persistent memory so the agent can recall context across sessions, not just within a single conversation.
- Build a simple planning step, where the agent breaks a task into sub-steps before executing, instead of responding in a single pass.
- Experiment with a basic multi-agent setup (e.g., a 'planner' agent and a 'critic' agent) — a concept I understand in theory but haven't built hands-on yet.
- Get comfortable with a framework like LangGraph or a similar orchestration tool, instead of writing all agent logic manually.
- Add proper logging/tracing so I can see exactly what the agent 'thought' and did at each step, not just the final output.
- Improve error handling for tool calls specifically — e.g., what happens if a tool fails or returns unexpected data.
- Write automated tests for at least the core logic, instead of only manually clicking through the app to check it works.
- Carry forward this week's debugging habit — reading the actual error and framework behavior carefully instead of guessing — into more complex agentic code.
- Document the agent's architecture as I build it, rather than writing it all up at the end, so the reasoning behind each design decision doesn't get lost.
- Explore how memory and tool use interact — e.g., should the agent decide on its own when to save something to memory versus being told explicitly.
- Start thinking about how these Week 2 concepts (tools, memory, planning) could eventually apply to my FYP's conversational agent.